UNIVERSITY OF ENGINEERING AND TECHNOLOGY, LAHORE
DEPARTMENT OF ELECTRICAL ENGINEERING

Programming For AI

FALL 2026

PROJECT:1

Flight and Airport Analytics (SkyLens)

Submitted by:
BS.AI Fall 2025 Section C

Muhammad Usman	2025-AI-167
Abdul Rehman	2025-AI-155
M Saim Akram	2025-AI-146
Taha Abdul Rehman	2025-AI-274

Instructor: Mr. Shaban Satti

Submission Date: OCT-26-2026

Abstract

This report presents Flight and Airport Analytics (SkyLens), a Python application that implements a complete ETL pipeline and a structured exploratory data analysis (EDA) on real flight data. Records are extracted from the `/v1/flights` endpoint of the AviationStack REST API, validated against a Pydantic schema, flattened and cleaned with pandas, and loaded without duplicates into a local SQLite database, with raw JSON and processed CSV copies kept as files. The EDA examines dataset structure, data quality, status, airline, airport and route distributions, delays and departure-time patterns. The same shared service layer feeds a numbered terminal menu with eleven actions and a responsive PySide6 desktop interface with nine pages. Long-running work runs on background threads, and analyses report "unavailable" instead of inventing values such as delay causes.

The project is verified by an automated suite of 65 tests, all of which passed when executed for this report. The bundled database of 133 flights was analysed by the application itself: 81 airlines, 75 routes, 76 airports and 9 delayed flights (6.8%), with a mean delay of 2.5 min over the 66 flights that report one. Because 99.2% of the flights were still scheduled and half carry no delay value, the findings describe a small snapshot rather than air traffic in general. The report covers dataset choice, architecture, ETL methodology, validation, storage, EDA, testing, limitations and a procedure for packaging the project.

.

Table of Contents

1 Introduction **1**
1.1 Background and Motivation 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope 1
2 Dataset Selection and Description **1**
2.1 Choice of API and Dataset 1
2.2 Data Extracted 2
2.3 Storage Destination and Justification 2
3 Tools and Technologies **2**
4 System Architecture **3**
4.1 Layered Architecture 3
4.2 Module Responsibilities 4
4.3 Design Decisions 4
5 ETL Methodology **4**
5.1 End-to-End Workflow 4
5.2 Development Methodology 5
6 Data Extraction from the API **5**
6.1 Request Construction 5
6.2 Error Handling 5
7 Data Transformation and Validation **6**
7.1 Schema Validation with Pydantic 6
7.2 Flattening and Cleaning with pandas 7
7.3 Derived Features 7
8 Data Loading and Storage **7**
8.1 Database Schema 7
8.2 Idempotent Loading 8
9 Application Interfaces **9**
10 Exploratory Data Analysis Design **10**
10.1 Analytical Objectives and Questions 10
10.2 Method Notes and Scope 11
11 Results and Discussion **11**
11.1 Dataset Structure and Data Quality (Q1, Q2) 11
11.2 Distributions and Groupings (Q3, Q4, Q6, Q7) 12
11.3 Delay Analysis (Q5) 13
11.4 Summary of Findings 14
12 Testing and Verification **14**
13 Challenges and Limitations **15**
13.1 Challenges and Solutions 15
13.2 Limitations 16
14 Conclusion and Future Work **16**
References **16**

1 Introduction

1.1 Background and Motivation

Air travel generates a continuous stream of operational data: scheduled and actual times, delays, terminals, gates, aircraft identifiers and live positions. Commercial providers expose this data through web APIs, but a raw JSON response is nested, partly null and not directly suitable for analysis. Turning it into a trustworthy dataset requires an Extract–Transform–Load (ETL) process followed by systematic exploratory data analysis (EDA).

Flight and Airport Analytics (branded SkyLens in the desktop window) is a Python application that implements this complete cycle on real flight records: **AviationStack REST API → Pydantic validation → pandas transformation → SQLite storage → EDA, statistics and Matplotlib charts**. Two further observations motivated the design. The free API plan allows roughly 100 requests per month, so every downloaded record must be stored and reused. Many analytics tools also hide data-quality problems, whereas this project deliberately exposes them through validation reports, missing-value counts and explicit "data unavailable" messages.

1.2 Problem Statement

Raw flight data returned by a web API cannot be analysed conveniently by a non-programmer, and it cannot be trusted blindly: records may be malformed, many fields are null, free-tier plans limit the number of requests, and repeated downloads create duplicates. A user therefore needs a pipeline that (i) fetches the data safely, (ii) rejects invalid records explicitly instead of silently repairing them, (iii) stores results so that earlier downloads remain available when the quota is exhausted, and (iv) summarises the stored data without unsupported claims, for example about the cause of a delay.

1.3 Objectives

- **Extract** flight records from the AviationStack `/v1/flights` endpoint with optional filters, archiving each raw response.
- **Transform and validate** every record with Pydantic models, report exactly which records were rejected and why, and flatten the nested JSON into a clean pandas DataFrame with derived columns (`route`, `is_delayed`, `max_delay_min`).
- **Load** flights, derived airports and a refresh history into SQLite without duplicate rows on repeated retrievals, with processed CSV copies kept as files.
- **Analyse** the stored dataset through a structured EDA covering structure, data quality, distributions, delays and departure-time patterns, presented as statistics and eight chart types.
- Expose the same service layer through a terminal menu and a responsive desktop GUI, keep the API key protected, and verify the code with an automated test suite.

1.4 Scope

In scope are real-time flight retrieval, validation, cleaning, local storage, derivation of an airports table from the flight records, EDA, chart generation, CSV/JSON export and an optional informational weather lookup (Open-Meteo). Out of scope are historical flight databases, delay-cause prediction, machine-learning models, multi-user or server deployment and the paid AviationStack endpoints such as `/v1/airports`. The application is designed for a single user

on a desktop computer. This report documents the current root version of the project; the archive also holds an older snapshot (`pr1-2`) with a single-file GUI, which is not described further.

2 Dataset Selection and Description

2.1 Choice of API and Dataset

The dataset is the live-flight feed of the AviationStack API (`/v1/flights`) [1]. It was chosen for five reasons. First, it is real operational data rather than a prepared teaching dataset, so it contains the missing values, inconsistent blocks and duplicates that an ETL pipeline must handle. Second, it is delivered as JSON over REST with a simple access-key query parameter, and a free plan is available to students. Third, each record is deeply nested (departure, arrival, airline, flight, aircraft and live blocks), which makes flattening and schema validation meaningful rather than trivial. Fourth, the records mix categorical (status, airline, airport codes), numeric (delay minutes, speed), temporal (scheduled, estimated and actual times), boolean (codeshare) and geographic (latitude, longitude) fields, which supports a varied EDA. Fifth, a natural key (flight date, flight number, departure and arrival code) identifies a flight, which enables reliable de-duplication.

The trade-offs of the free plan are accepted and documented: about 100 requests per month, HTTP only, no historical data, no delay causes and no airport directory. They shape the design (local storage, derived airports, no causal claims). Open-Meteo [2] is used only as an optional, key-less source of current weather for context; it is not part of the stored dataset.

2.2 Data Extracted

Each API response contains a `pagination` object (limit, offset, count, total) and a `data` array of flight records. The three raw responses archived in the project report a total of 10,000 matching records, of which 100, 25 and 10 were requested. After flattening, every flight becomes one row with 41 data columns (Table 1). The bundled database used for all results in this report holds 133 flights with flight dates from 2026-10-09 to 2026-10-10.

Table 1: Structure of a flight record and its role in the analysis

Record block	Representative fields	Role in the analysis
Flight header	`flight_date`, `flight_status` (six allowed values)	Natural-key component; status distribution
Departure / arrival	airport name, IATA and ICAO codes, timezone, terminal, gate, delay, scheduled / estimated / actual times (11 departure and 12 arrival columns)	Airport, route, delay and time analysis
Airline and flight	airline name, IATA, ICAO; flight number, flight IATA/ICAO, codeshare details	Airline ranking; codeshare share; natural key
Aircraft (optional)	registration, ICAO24	Descriptive identifiers, frequently null
Live (optional)	latitude, longitude, horizontal speed and other tracking fields (5 columns)	Live-position chart; present only for tracked aircraft
Derived by the pipeline	`route`, `is_delayed`, `max_delay_min`, codeshare flag	Features used directly by the EDA

2.3 Storage Destination and Justification

Processed data is loaded into a **SQLite database** (`database/flight_airport.db`). SQLite is a self-contained, serverless SQL engine stored in one file [5] and accessed through the standard `sqlite3` module [12]. It suits a single-user desktop tool because it needs no server installation, supports SQL filtering for the explorer pages, and offers a `UNIQUE` constraint that enforces de-duplication in the database itself. Because the database persists between runs, previously downloaded records stay available when the monthly quota is exhausted. Two file stores complement it: the **untouched raw JSON** of every response (`data/raw/`) for reproducibility, and **processed CSV** snapshots and exports (`data/processed/`) for portability. A client–server database was not considered necessary for a single-user scope.

3 Tools and Technologies

The application is implemented in pure Python 3.10 or later (verification used Python 3.12.3) with type hints and `pathlib` paths, and runs on Windows, macOS and Linux. Table 2 lists each technology and its role. A free AviationStack account and access key are required for retrieval; no key is needed for Open-Meteo.

Table 2: Technologies used and their roles

Technology	Version	Role in the project
Python [8]	≥ 3.10	Implementation language for all modules
requests [10]	≥ 2.31	HTTP client for AviationStack and Open-Meteo
python-dotenv [11]	≥ 1.0	Loads configuration from `.env` (utf-8-sig encoding tolerates a Windows byte-order mark)
Pydantic v2 [4]	≥ 2.5	Structural and range validation of API records at the system boundary
pandas [3]	≥ 2.0	Flattening, cleaning, grouping, descriptive statistics, CSV/JSON export
SQLite / sqlite3 [5], [12]	standard library	Local persistent storage with a three-table schema
Matplotlib [6]	≥ 3.8	Eight chart builders; the same figures are embedded in the GUI and saved as PNG
PySide6 (Qt 6) [7]	≥ 6.6	Desktop GUI: widgets, table model, thread pool, signals, stylesheets
pytest [9]	≥ 8.0	Automated unit, headless GUI and regression tests
AviationStack API [1]	free plan	Primary real-time flight data source (`/v1/flights`)
Open-Meteo API [2]	no key	Optional weather context, never used to explain delays

4 System Architecture

4.1 Layered Architecture

The application has three layers (Figure 1). The **presentation layer** contains the two entry points, the terminal menu (`main_task/main.py`, 11 actions) and the PySide6 GUI (`main_task/gui_main.py`). The **application layer** contains the shared service, the analytics module and the chart module. The **data layer** contains the API client, the Pydantic models, the pandas processor and the SQLite access module. The cross-cutting modules

`config.py`, `utils.py` and `palette.py` supply settings, logging, exception types and colour tokens to all layers.

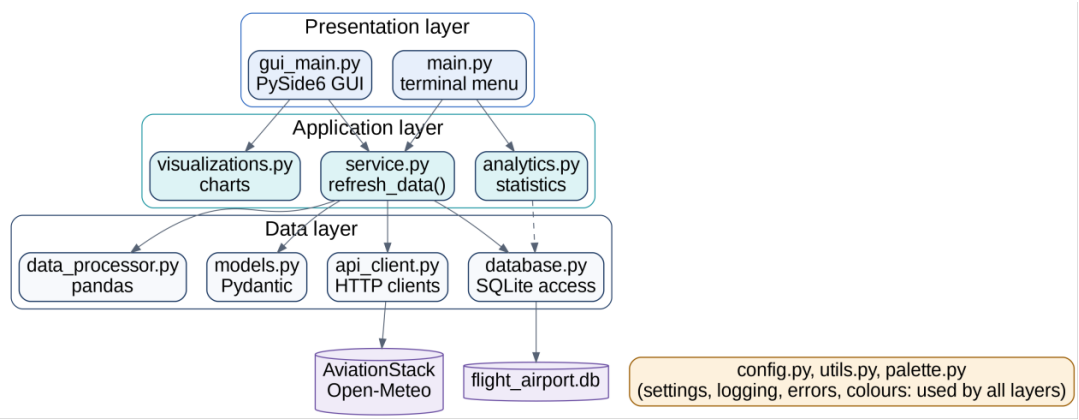


Figure 1: Layered architecture of SkyLens

4.2 Module Responsibilities

Table 3 maps each source module to its ETL stage. The `src` package and entry points total roughly 5,000 lines of Python.

Table 3: Source modules and their position in the ETL pipeline

Module	Lines	Stage and responsibility
`config.py`	158	All stages: paths, `.env` loading, settings, API-key masking, safe `.env` writing
`utils.py`	122	All stages: exception hierarchy, rotating log file (1 MB, three backups), helpers
`api_client.py`	221	**Extract**: HTTP requests, raw-response archiving, error mapping, weather lookup
`models.py`	165	**Transform**: Pydantic schema and batch validation
`data_processor.py`	271	**Transform**: flattening, cleaning, derived columns, airport derivation, quality report
`database.py`	403	**Load**: schema, upserts, filtered queries, CSV/JSON export
`service.py`	120	Orchestration: `refresh_data()` runs the full cycle and returns a `RefreshResult`
`analytics.py`	289	**EDA**: statistics and display classification over DataFrames
`visualizations.py`	307	**EDA**: eight Matplotlib chart builders, PNG saving
`gui/` package, `main.py`	2,362 / 500	Presentation: nine-page desktop interface and terminal menu

4.3 Design Decisions

- **Single orchestration point.** `service.refresh_data()` is called identically by both front-ends, so they behave the same way.
- **Validation before storage.** Records failing Pydantic validation are counted and reported but never stored; the rest of the batch continues and the refresh is logged as `partial`.
- **Natural-key de-duplication.** A flight is identified by `(flight_date, flight_iata, dep_iata, arr_iata)`; a repeated retrieval replaces the old row.

- **Derived airports.** The paid airport directory is not assumed, so airports are derived from the departure and arrival blocks and nothing is invented.
- **No unsupported inference.** Weather is never blamed for a delay unless the source confirms it, and analyses without data return an explanatory message instead of numbers.
- **Responsive GUI.** Network work runs on a Qt thread pool (maximum four threads) and results return to the GUI thread through signals.

5 ETL Methodology

5.1 End-to-End Workflow

The complete cycle is implemented by `service.refresh_data()` and triggered from the GUI Refresh button (F5), the Data Management page or terminal option 2 (Figure 2). If the API key is missing, the GUI warns the user and opens the Settings page. The request is sent and the raw JSON is archived. An error payload or HTTP failure raises a specific exception that is shown as a readable message, and a response with zero records is logged as `empty`. Otherwise the records are validated, flattened, cleaned and de-duplicated, airports are derived, flights and airports are upserted into SQLite, the processed CSV is saved and the attempt is written to the refresh log. The GUI then invalidates its cached data store, reloads the current page and updates the status bar.

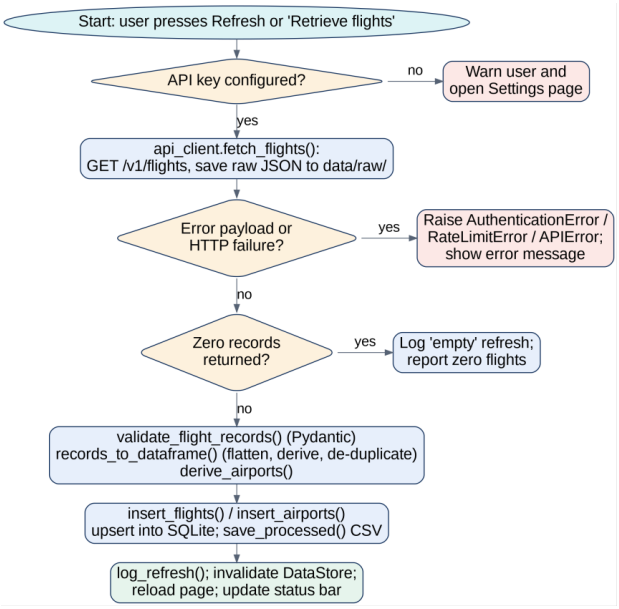


Figure 2: ETL workflow implemented by `refresh_data()`

5.2 Development Methodology

Development was incremental and test-supported. The data pipeline (client, models, processor, database) was built first as a command-line tool; analytics and charts were added next; the GUI was layered on top of the same modules last. The schema was designed from the structure of the archived `/v1/flights` response. Analytics are pure functions over DataFrames, so both front-ends reuse them and tests can call them directly. The result of each ETL run is summarised in a `RefreshResult` dataclass (records fetched, valid, invalid and stored, airports stored, validation error messages and the processed file path).

6 Data Extraction from the API

6.1 Request Construction

`fetch_flights()` sends `GET {base_url}/flights` with the `access_key` parameter and the optional filters `limit`, `offset`, `flight_status`, `dep_iata`, `arr_iata`, `airline_name` and `flight_number`. The record limit is clamped between 1 and 1000, and a status filter is only sent if it belongs to the six known values. The base URL defaults to `http://api.aviationstack.com/v1` because the free plan does not support HTTPS; the timeout defaults to 20 s and the default record limit to 100. All settings come from `.env`, which is excluded from version control, and the key is shown only in masked form.

Before any transformation, the response is saved unchanged to `data/raw/flights_raw_<timestamp>.json`. This preserves an untouched copy of the source, so a transformation bug can never destroy the original data and the quota is not wasted.

6.2 Error Handling

`_request()` parses the JSON body even for 4xx responses, so that the provider's explanation is shown instead of a bare status code. `_handle_error_payload()` then maps provider error codes to specific exception types (Table 4).

Table 4: Mapping of API failures to application behaviour

Situation	Application behaviour
No key configured; codes `missing_access_key`, `invalid_access_key`, `invalid_api_key`, `101`; HTTP 401	`AuthenticationError` with instructions to configure `.env`
Codes `usage_limit_reached`, `rate_limit_reached`, `104`, any code containing "limit"; HTTP 429	`RateLimitError` (free plan: about 100 requests per month)
`function_access_restricted`, `https_access_restricted`, `not_found`	`APIError`: endpoint not available on this plan
Timeout, connection error, other HTTP status ≥ 400, invalid JSON, missing `data` key	`APIError` with a readable message

```
payload = response.json()   # parsed first; None if the body is not JSON
_handle_error_payload(payload)   # raises a specific error type
if response.status_code == 401:
    raise AuthenticationError("API returned 401 - the access key is invalid.")
if response.status_code == 429:
    raise RateLimitError("API returned 429 - rate limit reached, try later.")
if response.status_code >= 400:
    raise APIError(f"API returned HTTP {response.status_code}.")
```

The optional weather lookup `fetch_weather()` requests `temperature_2m`, `weather_code` and `wind_speed_10m` from Open-Meteo and returns `None` on any failure, so this secondary source can never break the main workflow.

7 Data Transformation and Validation

7.1 Schema Validation with Pydantic

Validation is applied at the system boundary so that malformed external data never reaches the database. `models.py` mirrors the `/v1/flights` structure: `FlightRecord` contains

`departure` and `arrival` (`AirportPointInfo`), `airline`, `flight` and the optional `aircraft` and `live` blocks (Figure 3). Fields that the API often returns as null are declared `Optional`, so null alone is not an error. The enforced rules are listed in Table 5, and the status validator is shown below.

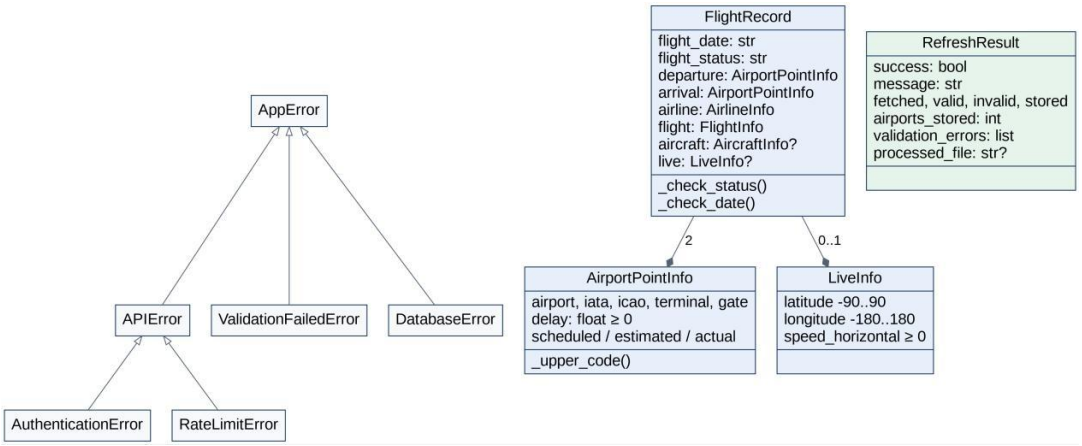


Figure 3: Exception hierarchy, validation models and `RefreshResult`

```
VALID_FLIGHT_STATUSES = {"scheduled", "active", "landed",
                         "cancelled", "incident", "diverted"}


class FlightRecord(BaseModel):
    flight_date: str = Field(min_length=8, max_length=12)
    flight_status: str
    departure: AirportPointInfo
    arrival: AirportPointInfo
    @field_validator("flight_status")
    @classmethod
    def _check_status(cls, value: str) -> str:
        status = (value or "").strip().lower()
        if status not in VALID_FLIGHT_STATUSES:
            raise ValueError(f"unrecognised flight status {value!r}")
        return status
```

`validate_flight_records()` validates a whole batch and returns the valid records together with readable messages for the rejected ones. Rejected records are counted and listed in the `RefreshResult`; they are not repaired, which keeps the dataset honest.

7.2 Flattening and Cleaning with pandas

`records_to_dataframe()` converts each validated record into a flat dictionary with the 41 columns of `FLIGHT_COLUMNS`, and `clean_flights()` applies the cleaning steps in Table 5. The same natural key is used twice: in pandas (`drop_duplicates(keep="last")`) to remove duplicates inside one response, and in SQLite to replace older rows when the same flight is retrieved again.

Table 5: Validation (Pydantic) and cleaning (pandas) rules

Stage	Rule enforced
Pydantic: values	Status must be one of the six known values (case-normalised); date must be a valid ISO date string; delays ≥ 0; latitude in [−90, 90]; longitude in [−180, 180]; speed ≥ 0
Pydantic: format	IATA/ICAO codes are trimmed and upper-cased (for example “ jfk ” becomes JFK)
Pydantic: structure	Blocks `departure`, `arrival`, `airline` and `flight` are required; `aircraft` and `live` are optional; null values are accepted where the API commonly returns them
pandas: types	Delay columns converted to numbers; six timestamp columns parsed as UTC datetimes; text columns trimmed
pandas: rows	Rows with neither a departure nor an arrival code dropped; duplicates on the natural key removed, keeping the last occurrence

7.3 Derived Features

Three analysis columns are derived for every row. `route` joins departure and arrival codes (for example BKK-SAI). `max_delay_min` is the larger of the departure and arrival delays, and `is_delayed` is true when it is greater than zero. A codeshare flag is derived from the flight block. `derive_airports()` stacks the departure and arrival blocks, groups them by IATA code and counts departures, arrivals and total sightings. Airport records therefore contain only fields present in the flights (name, codes, time zone, counts) and no coordinates. Finally, `quality_report()` summarises rows, columns, missing values and duplicates for the EDA.

8 Data Loading and Storage

8.1 Database Schema

`database.py` is the only module that touches SQLite. The schema (Figure 4, Table 6) is created the first time a connection is opened and consists of three tables and four indexes. Foreign-key enforcement is switched on, but the schema declares no foreign keys: the link between `flights` and `airports` through IATA codes is logical only.

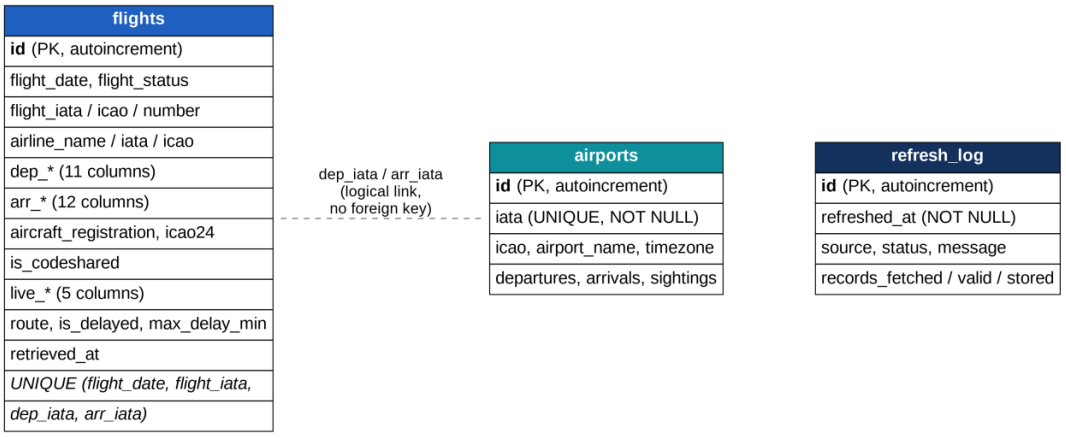


Figure 4: Entity-relationship diagram of the SQLite database

Table 6: Database tables and constraints

Table	Content	Keys and constraints
`flights`	One row per flight: 41 data columns plus `id` (identifiers, airline, departure and arrival details, delays, timestamps, aircraft, live position, derived fields)	`UNIQUE(flight_date, flight_iata, dep_iata, arr_iata) ON CONFLICT REPLACE`; indexes on status, airline, departure and arrival codes
`airports`	Airports observed in the flights, with departure, arrival and total counts	`iata` UNIQUE NOT NULL; upsert with `ON CONFLICT(iata) DO UPDATE`
`refresh_log`	One row per retrieval attempt: time, source, fetched / valid / stored counts, status, message	`refreshed_at` NOT NULL

8.2 Idempotent Loading

A repeated retrieval must not create duplicates. The `UNIQUE` constraint with `ON CONFLICT REPLACE` guarantees this, with one subtlety: SQLite treats NULL values as distinct inside a unique constraint, so records with a missing airport code would never be recognised as duplicates. `insert_flights()` therefore converts missing key columns to an empty string before writing.

```
UNIQUE(flight_date, flight_iata, dep_iata, arr_iata) ON CONFLICT REPLACE

# insert_flights(): NULLs are distinct in UNIQUE constraints in SQLite,
# so key columns are coerced to '' to keep de-duplication reliable.
for col in columns:
    if col in UNIQUE_KEY and values[idx] is None:
        values[idx] = ""
```

All access goes through a `connect()` context manager that commits on success, rolls back on error and always closes the connection. Queries are parameterised, and column and table names used in dynamic SQL are checked against `FLIGHT_COLUMNS` and an allow-list, which prevents SQL injection through those parameters. Every attempt, including failures, is recorded in `refresh_log`. The module also exports flights and airports to CSV and JSON and the analytics summary to JSON.

9 Application Interfaces

Both front-ends present the stored data without containing business logic. The terminal menu offers 11 actions (overview, retrieve data, view flights, view airports, search and filter, statistics, exploratory data analysis, generate visualisations, database information, export and configuration status) with validated input. The PySide6 GUI (Figure 5) has nine pages: Overview, Flight Analytics, Airport Analytics, Flight Explorer, Delay Analysis, Weather Insights, Data Management, Settings and About. It also provides a header search, a Refresh button, a status bar with colour-coded flight-condition chips, keyboard shortcuts (Ctrl+1–9, F5, Ctrl+K), a sidebar that collapses on narrow windows and a Settings page that stores the API key in `.env` and displays it only masked.

Responsiveness is achieved by a `TaskRunner` that executes network and database work on a `QThreadPool` and returns results to the GUI thread through signals; callbacks for destroyed widgets are skipped. A cached `DataStore` prevents pages from re-querying SQLite, and search boxes use a 180 ms debounce. The screenshot below was captured from the real

application with the bundled database; “API key missing” is shown because the `.env` file was deliberately not copied into the capture environment, and no live request was made.

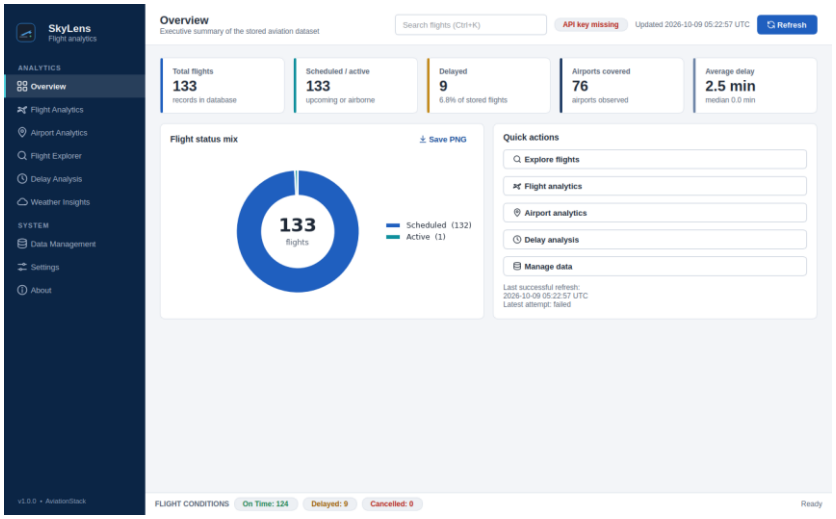


Figure 5: Overview page of the GUI: key metrics, status donut chart and quick actions

The Flight Explorer page (Figure 6) combines free-text search, a status selector, origin and destination IATA fields and a “Delayed only” check box over a sortable table; the label above the table shows how many stored flights match.

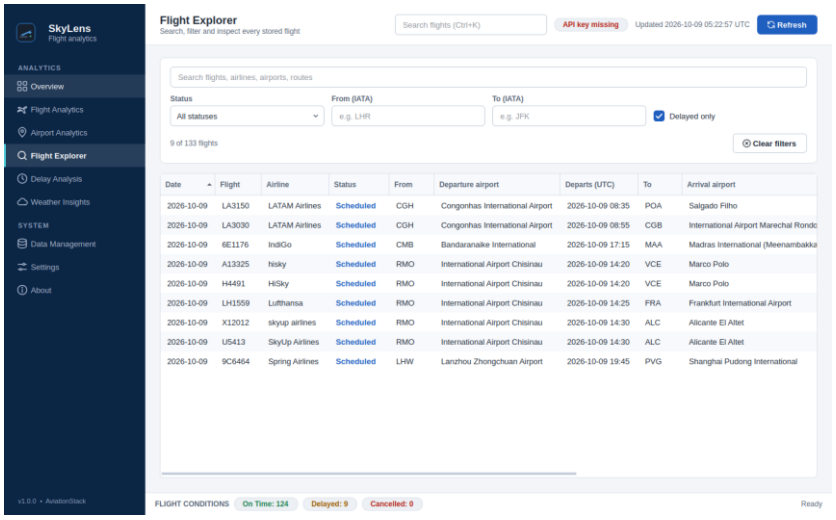


Figure 6: Flight Explorer page with the “Delayed only” filter enabled

Figure 7 shows the same refresh as a sequence of calls: the GUI submits `refresh_data()` to the worker thread, the service performs the request, validation, transformation and database writes, and the completion callback is delivered back on the GUI thread, which then reloads the page and updates the status bar.

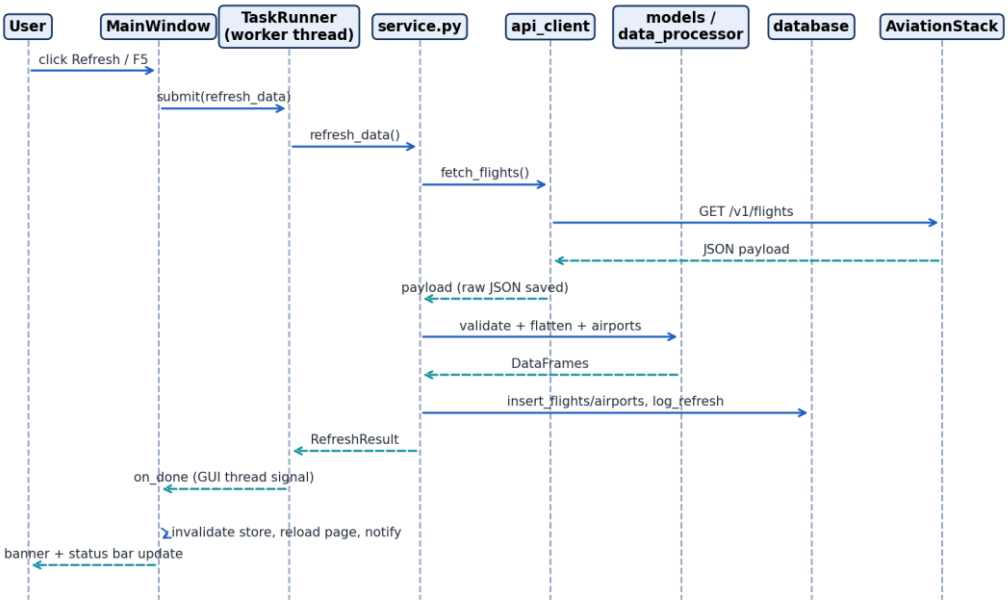


Figure 7: Sequence diagram of a refresh started from the GUI

10 Exploratory Data Analysis Design

10.1 Analytical Objectives and Questions

The EDA is organised as four stages: structural profiling, data-quality assessment, univariate distributions and grouped (relational) comparisons. Table 7 states the question each analysis answers and the code that implements it. All analytics in `analytics.py` are pure functions that accept a DataFrame and return plain structures, so the terminal, the GUI and the tests share one implementation.

Table 7: EDA questions, techniques and implementation

#	Analytical question	Technique	Implementation
Q1	What are the size, columns and data types of the dataset?	Structural profiling, unique-value counts	`dataset_overview`, `unique_value_counts`, `numeric_summary`
Q2	How complete and clean is the data (missing values, duplicates, invalid records)?	Missing-value and duplicate counts; validation report	`quality_report`, `RefreshResult`, `refresh_log`
Q3	What is the mix of flight statuses?	Categorical frequency	`status_distribution`, donut chart
Q4	Which airlines, airports and routes dominate?	Top-N value counts and grouping	`top_airlines`, `top_airports`, `top_routes`, bar charts
Q5	How common and how large are delays?	Count, share, mean, median, maximum; histogram	`delay_statistics`, delay histogram
Q6	At what hours do flights depart?	Temporal binning (hour of day, UTC)	`departure_hour_distribution`, bar chart
Q7	How common are codeshares, and are live positions available?	Share of a boolean flag; completeness of a block	`codeshare_share`, live-position chart

10.2 Method Notes and Scope

Delay statistics use all rows that have a delay value, whereas the delayed share is computed against all stored rows; when no delay values exist, `delay_statistics()` returns `available: False` with a reason instead of numbers. Each chart builder returns an explanatory "empty" figure when the data cannot support the plot. Relationships between variables are explored through grouped counts (routes as origin–destination pairs, departures against arrivals per airport). Correlation analysis, hypothesis tests, time-series trends and maps were **not** implemented: the stored data spans only two days, is 99.2% scheduled and the derived airports have no coordinates, so such analyses would not be supported by the data.

A separate display classification, `classify_condition()`, labels each row for the status-bar chips by applying rules in a fixed order: cancelled; delayed (labelled a weather delay only if the source confirms it); weather alert only if confirmed; incident or diverted; scheduled or active as "On Time"; landed; otherwise "Status Unknown". A regression test checks that the vectorised `condition_counts()` agrees with the row-by-row classifier.

```
if status == "cancelled":
    return {"label": "Cancelled", "color": palette.ERROR}
if delay > 0:
    if weather_confirmed:
        return {"label": f"Weather Delay {int(delay)} min", ...}
    return {"label": f"Delayed {int(delay)} min", "color": palette.WARNING}
...
if status in {"scheduled", "active"}:
    return {"label": "On Time", "color": palette.SUCCESS}
```

11 Results and Discussion

All results in this section were computed by the application from the bundled database; no live API request was made while preparing this report.

11.1 Dataset Structure and Data Quality (Q1, Q2)

Table 8: Structure and data-quality indicators of the bundled dataset

Indicator	Value
Flights / airports / refresh-log rows	133 / 76 / 4
Columns	41 data columns plus `id` (42 in the CSV export)
Flight dates covered	2026-10-09 to 2026-10-10
Records fetched / valid / rejected	135 / 135 / 0 (three successful refreshes)
Duplicates replaced on load	2 (135 records written, 133 rows stored)
Flights with a delay value	66 of 133; the other 67 (50.4%) have none
Live-tracked flights	0 (all live columns empty)

Table 9: Refresh history stored in `refresh_log`

#	Time (UTC)	Fetched / valid / stored	Status
1	2026-10-09 05:15:17	100 / 100 / 100	success
2	2026-10-09 05:21:10	25 / 25 / 25	success
3	2026-10-09 05:22:57	10 / 10 / 10	success
4	2026-10-11 06:59:06	0 / 0 / 0	failed: "API returned HTTP 403."

The dataset has a mixed schema: categorical identifiers (status, airline, airport codes), numeric delay values, six UTC timestamp columns, boolean flags and the optional live block. The pipeline accepted every record that the API returned: all 135 records passed validation. The rejection path was therefore exercised only by the automated tests (Section 12), not by live data. The three successful refreshes wrote 135 records but only 133 rows are stored, which is consistent with the natural-key rule replacing two repeated flights.

Completeness is the main quality finding. Half of the flights (67 of 133) carry no delay value and none carries live-position data, so delay statistics describe only the 66 flights that report one. The fourth refresh failed with HTTP 403 and stored nothing, and the database remained intact, which demonstrates the intended failure behaviour.

11.2 Distributions and Groupings (Q3, Q4, Q6, Q7)

Table 10: Results of the distribution analyses

Measure	Result
Status mix (Q3)	scheduled 132 (99.2%), active 1 (0.8%); condition chips: On Time 124, Delayed 9, Cancelled 0
Distinct airlines / routes	81 / 75; busiest airlines China Eastern 6, Sichuan Airlines 6, China Southern 5
Distinct airports (Q4)	20 departure and 64 arrival airports (76 in total)
Busiest departure airports	TFU 28, BKK 14, LHW 13
Busiest arrival airports	SAI 16, CMB 12, LFQ 12
Most frequent routes	BKK-SAI 14, DOH-CMB 11, TSN-LFQ 8
Codeshared flights (Q7)	56 (42.1%)

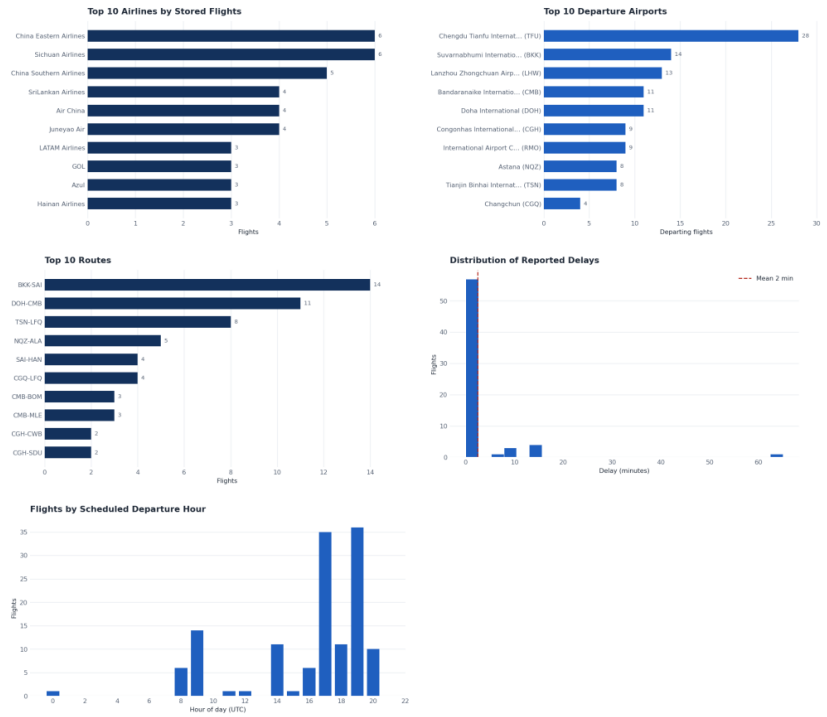


Figure 8: Charts generated by `save_chart()` from the stored dataset: top airlines, top departure airports, top routes, delay distribution and departures by hour (UTC)

Status. Almost all flights are still “scheduled”. The three successful retrievals took place between 05:15 and 05:23 UTC on 2026-10-09, before nearly all departure times, so the dataset

is a snapshot of planned flights rather than completed operations. Conclusions about on-time performance would be premature.

Airlines, airports and routes. The 133 flights involve 81 airlines, so the distribution has a long tail: even the leaders hold only 5–6 flights each (Figure 8). Departures are far more concentrated than arrivals: 20 departure airports against 64 arrival airports, with Chengdu (TFU) alone accounting for 28 flights (21.1%) and the single route BKK-SAI for 14 (10.5%). The leading airlines are largely Chinese carriers. This concentration most likely reflects how the free-plan response was ordered and sampled rather than real traffic shares, but the archive does not allow this cause to be confirmed; the statistics must be read as describing this sample only.

Departure hours. Departures are strongly bimodal in UTC. Two bars dominate, at 17 h and 19 h, with roughly 35 and 36 flights read from the chart (about 53% of all flights together); smaller groups appear at 8–9 h and 14 h. Local-time interpretation would require the airport time zones, which are stored but not applied in the chart.

Codeshares. About 42% of the records are codeshared flights. Because each marketing flight number is stored as its own row, a physical flight may appear more than once, which can inflate airline counts. The codeshare flag is available to filter such rows in later analysis.

11.3 Delay Analysis (Q5)

Table 11: Delay statistics (66 flights with a delay value)

Measure	Value
Delayed flights (`is_delayed`)	9, i.e. 6.8% of all 133 stored flights (13.6% of the 66 flights with a delay value)
Average / median / maximum delay	2.5 min / 0.0 min / 65 min

The delay distribution is strongly right-skewed (Figures 8 and 9): about 57 of the 66 flights report zero delay, a few report 5–15 minutes and one reaches 65 minutes. This single outlier lifts the mean (2.5 min) above the median (0.0 min), so the median is the more representative figure and the mean should be quoted with it. Because 67 flights have no delay value and are counted as not delayed, the 6.8% share is a conservative lower bound on the true proportion. The free API tier supplies no delay causes, so the application reports delay minutes only and never attributes a delay to weather.

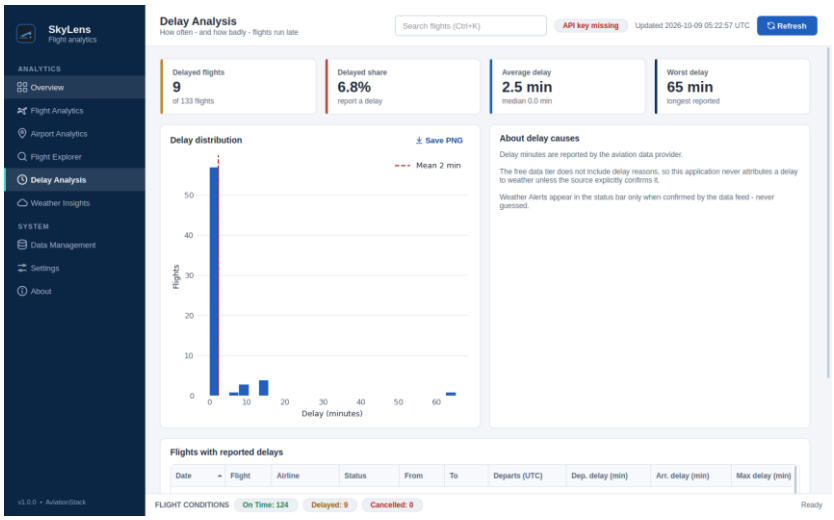


Figure 9: Delay Analysis page of the GUI: delayed share, average and worst delay, delay histogram and a note that delay causes are not provided

11.4 Summary of Findings

- The ETL pipeline retrieved, validated and stored 135 records without rejection, ignored two duplicates and survived a failed request without data loss.
- The stored dataset is small and skewed (99.2% scheduled, half without delay values, no live data), so descriptive findings are valid for the sample but not for air traffic in general.
- Flights are concentrated in a few departure airports, routes and evening UTC hours, and delays are rare and mostly zero with one large outlier.

12 Testing and Verification

Testing combines unit tests, headless GUI tests and regression tests executed with pytest; none needs network access or an API key. A shared fixture supplies three AviationStack-shaped sample records (an active JFK→LAX flight with live position and delays, a cancelled codeshared ORD→LAX flight and a landed JFK→MIA flight), and database tests use temporary SQLite files so the real database is never modified. The complete suite was executed for this report on a copy of the project using Python 3.12.3, pytest 9.1.1, PySide6 6.12.0, pandas 3.0.2 and Pydantic 2.14.1: **65 tests collected, 65 passed, 0 failed in 9.31 s**.

Table 12: Automated test modules

Module	Tests	Focus
`test_validation.py`	9	Statuses, dates, missing blocks, negative delays, coordinate range, batch validation, code normalisation
`test_data_processor.py`	11	Flattening, derived columns, codeshare flag, de-duplication, timestamps, airport derivation, quality report
`test_database.py`	8	Insert and retrieve, no duplicates on refresh, filters, airports, refresh log, CSV/JSON export
`test_analytics.py`	13	Overview, status, top lists, delay statistics (including empty data), hours, charts on normal and empty frames
`test_gui_smoke.py`	15	Every page on empty and filled databases, thread callbacks, sorting, filters, refresh outcomes, settings validation
`test_regressions.py`	9	Fixed bugs: `.env` byte-order mark, `.env` writing, API error bodies, connection closing, schema rebuild

Module	Tests	Focus
Total	**65**	

Table 13: Representative test cases and expected results

ID	Test	Input / action	Expected result
TC-02	`test_unknown_status_is_rejected`	Status “teleported”	`ValidationError`
TC-04	`test_negative_delay_rejected`	Departure delay −5	`ValidationError`
TC-05	`test_invalid_latitude_rejected`	Latitude 999	`ValidationError`
TC-06	`test_batch_validation_separates_good_from_bad`	3 good and 2 bad records	3 valid records, 2 error messages
TC-07	`test_iata_codes_are_normalised`	IATA “ jfk ”	Stored as JFK
TC-09	`test_refresh_does_not_duplicate`	Insert the same 3 flights twice	Still 3 rows
TC-12	`test_env_file_with_bom_is_read`	`.env` saved with UTF-8 BOM	API key recognised
TC-15	`test_http_error_surfaces_the_api_message`	HTTP 403 with provider error body	Message shows the provider reason
TC-19	`test_condition_counts_match_row_classifier`	Mixed statuses and delays	Vectorised counts equal row-by-row

Representative assertions confirm the ETL behaviour: a status “teleported”, a non-date, a negative delay or a latitude of 999 raise a `ValidationError`; a batch of three good and two bad records yields three valid records and two error messages; inserting the same three flights twice still leaves three rows; and the vectorised condition counts equal the row-by-row classifier. The tests use fixture data, so they verify the logic but not the live API.

13 Challenges and Limitations

13.1 Challenges and Solutions

Table 14: Challenges encountered and their solutions

Challenge	Solution
`.env` saved on Windows begins with a byte-order mark, so the API key appeared unset	`.env` is read with `utf-8-sig` and rewritten as plain UTF-8; two regression tests
The provider explains failures in the JSON body, even on HTTP 403	Body parsed first and mapped to specific exceptions. The failed refresh of 2026-10-11 is still logged with the older message “API returned HTTP 403.”; the application log shows a request for 587 records, and the README states that free plans typically cap records at 100 per request, which is the probable (unconfirmed) cause

Challenge	Solution
About 100 requests per month and no history	Every response archived in `data/raw/` and stored in SQLite
Paid airport directory unavailable	Airports derived from the departure and arrival blocks
Repeated retrievals create duplicates; NULL keys defeat `UNIQUE`	Natural-key constraint with `ON CONFLICT REPLACE` and empty-string coercion
Network calls would freeze the GUI	`TaskRunner` thread pool with signals and an owner check
No delay causes available	Weather states appear only if confirmed by the data; weather is informational

13.2 Limitations

- **Data plan.** The free plan limits requests, provides no history or delay causes and uses HTTP only.
- **Small, skewed dataset.** 133 flights, 99.2% scheduled, no live-tracked aircraft; the EDA describes this snapshot only, and the live-position chart was empty.
- **Limited EDA depth.** No correlation, hypothesis testing or time-series analysis was performed (Section 10.2); delay causes cannot be analysed.
- **Derived airports.** No coordinates, so no route map is drawn; the airports table has no foreign-key link to flights.
- **Validation evidence.** Live data produced no rejected records, so the rejection path is verified only with fixtures.
- **Deployment.** Single-user design with SQLite and a local `.env`; no installer or executable is provided.
- **Documentation drift.** The README page table lists earlier page names that differ from the nine implemented pages.

14 Conclusion and Future Work

SkyLens implements a complete, layered ETL pipeline and EDA in Python. It extracts real flight records from AviationStack, rejects invalid records through Pydantic validation, flattens and cleans the data with pandas, loads it without duplicates into SQLite and analyses it through statistics and eight chart types, available from a terminal menu and a responsive PySide6 interface. Shared service logic, a natural-key constraint, raw-data archiving and explicit “data unavailable” messages make the pipeline robust and honest about the limits of a free API plan. All 65 automated tests passed, and the bundled database of 133 flights was analysed by the application itself (81 airlines, 75 routes, 76 airports, 6.8% delayed). The project therefore meets its stated objectives within the scope of a single-user desktop tool.

The following improvements are **proposals only** and are not features of the current code:

- Add scheduled automatic refreshes or a historical source to build a larger, time-based dataset within API quotas.
- Obtain airport coordinates from a reference dataset to draw route maps and compute distances.
- Extend the EDA with correlation and time-series analysis and, with enough data, statistical delay models that separate observation from prediction.

- Package the GUI as a Windows executable, add mocked-API integration tests and update the README page table.

References

[1] apilayer, “Aviationstack API documentation,” [Online]. Available: https://aviationstack.com/documentation

[2] Open-Meteo, “Free weather API documentation,” [Online]. Available: https://open-meteo.com/en/docs

[3] W. McKinney, “Data structures for statistical computing in Python,” in Proc. 9th Python in Science Conf. (SciPy 2010), 2010, pp. 56–61.

[4] Pydantic, “Pydantic documentation,” [Online]. Available: https://docs.pydantic.dev

[5] SQLite, “About SQLite,” [Online]. Available: https://www.sqlite.org/about.html

[6] J. D. Hunter, “Matplotlib: A 2D graphics environment,” Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.

[7] The Qt Company, “Qt for Python (PySide6) documentation,” [Online]. Available: https://doc.qt.io/qtforpython-6/

[8] Python Software Foundation, “Python 3 documentation,” [Online]. Available: https://docs.python.org/3/

[9] pytest-dev, “pytest documentation,” [Online]. Available: https://docs.pytest.org

[10] K. Reitz, “Requests: HTTP for Humans,” [Online]. Available: https://requests.readthedocs.io

[11] S. Kumar, “python-dotenv,” [Online]. Available: https://github.com/theskumar/python-dotenv

[12] Python Software Foundation, “sqlite3 — DB-API 2.0 interface for SQLite databases,” [Online]. Available: https://docs.python.org/3/library/sqlite3.html